

An Active Learning Approach to Predicting Potency of Small Molecules for Antibacterial Screening

Sriram Kidambi, Daniel Lee, Mirudhula Mukundan

Abstract

Drug discovery and antibiotic screening are time-consuming and cumbersome wet-lab procedures. Screening for potency in general involves several small molecules and working with each of them at the lab-scale is an expensive ordeal. Conducting computational analysis before the lab screening can not only decrease the number of potential target molecules but also reduce expenditure by a huge margin. In this project, we explore the usage of Passive and Active Machine Learning approaches to identify potent compounds against the bacterium, *Burkholderia cenocepacia*, that is commonly known to be an opportunistic pathogen, mainly affecting immunocompromised patients suffering from cystic fibrosis.

1 Introduction

1.1 Active Learning

Active learning is a technique used in machine learning that iteratively selects a subset of data from a large sample of unlabeled data to be labeled via a domain expert or oracle, and then uses the newly labeled points to train machine learning models. In this way, models can achieve a high performance while utilizing fewer labeled data. Within active learning, there are multiple methods and algorithms to select the data subset to be labeled, with each giving different performance results.

1.2 Significance

The significance of active learning lies in the efficient computational cost as the method does not require all of the data to be labeled. Labeling a data sample may cost hundreds of thousands of dollars and therefore active learning utilizes only the most important samples to label, thereby reducing financial/computational expenses while achieving high performance results.

1.3 Objective

The objective of this project is to analyze the power of active learning on a large dataset by exploring different known active learning methods and a novel Deep Reinforcement Learning method on a binary classification task. We compare their classification performance against one another, as well as the baseline random learning method.

2 Background

Computational approaches to drug discovery has been around for a while, but it is only in recent years that machine and deep learning approaches have been implemented to narrow down the search space and make appropriate decisions for potential drug molecules. Several research have been conducted using the molecular structures of the small molecules as input to machine learning models, either as graphs or as vectors. The different techniques used and the types of inputs to each model has been well reviewed by Carracedo-Reboredo *et al.* [1] and Jukic *et al* [5]. A popularly used method for drug discovery method named ChemProp uses a novel technique called Message-Passing Neural Networks [4], that utilizes Convolutional Neural Networks. Another research group has also performed Deep Learning techniques to discover molecules with bactericidal properties against *Mycobacterium tuberculosis* [10]. From recent research, a machine learning model was developed from the data obtained from high-throughput antibacterial screening to identify target molecules against *Burkholderia cenocepacia*, where they utilized ChemProp to model the data. As an off-shoot idea from their work, we used their dataset to run a simple machine learning model, but with active learning in order to train the model in such a way that minimum number of training samples is enough to achieve a good/desirable classification accuracy (or till the budget for conducting the experiments is finished). Utilization of active learning for drug discovery is still a new methodology to train models and a notable research by Naik and Kangas *et al* [6], was one of the first practical demonstration of applying active learning for high throughput search of small molecules and their effect on a target (here, protein).

3 Data

3.1 Source

The data was retrieved from the high-throughput screening conducted by Selin *et al.* where they tried to identify growth inhibitory compounds against the highly infectious *Burkholderia cenocepacia* [9, 8]. It contains the SMILES (Simplified molecular-input line-entry system) formatted data for 29,537 small molecules along with their average B-score value for each molecule. The authors describe the B-score value as a measure of antibiotic property that is inversely proportional to the potency of the compound against the bacterium. From their research, the authors specify the threshold for B-score to be at -17.5, i.e. a B-score lower than this threshold is said to be active against the *Burkholderia* strain. Staying true to the essence of their research, we will use the same threshold and binarize the data as 0 for inactive and 1 for active compound. We visualize the data with their respective labels, according to the set threshold, in Fig.1.

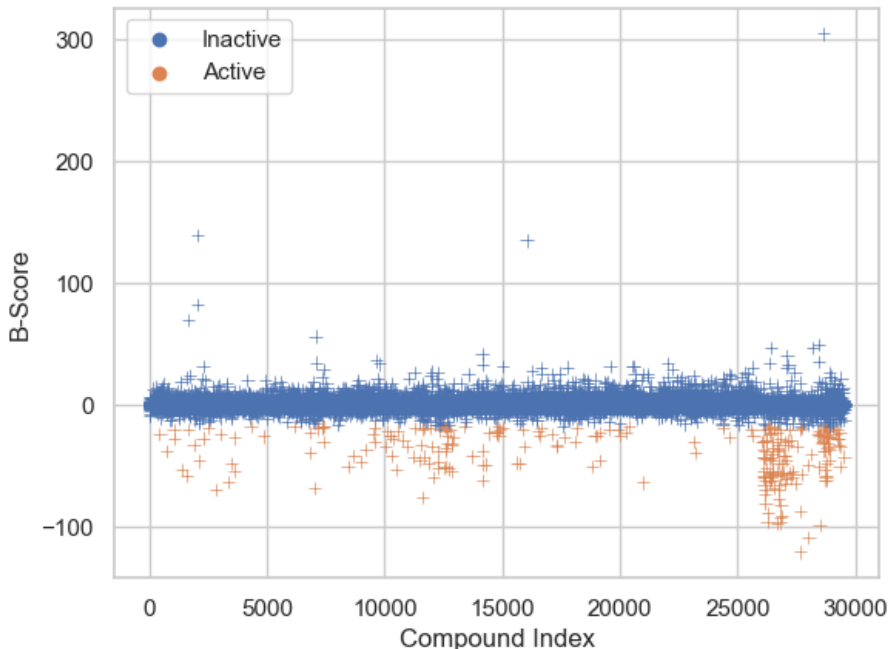


Figure 1: Visualization of all small molecules with respect to their Average B-score value. Blue crosses indicate B-score more than threshold value, and Red crosses indicate B-score lesser than the threshold value.

3.2 Representing SMILES Data as Molecular Fingerprints

Selecting an appropriate representation for the SMILES data is important in order to get informative features for downstream machine learning classification. One such method is to convert the molecular structure of

the compound to a 2048-bit array. By this method, we can obtain a feature set size that is unchanging for all samples. This was achieved by using the Python package RDKit.

3.3 Resolving Class Balance Issues

The two classes - active and inactive, are heavily unbalanced. We observed that the data had only 256 active compounds. For this reason, we first did uniform random undersampling of the major class, i.e. the inactive compounds. This way we had equal number of compounds in each class. We use this new class balanced data to run the models for 10 simulations. But, in order to eliminate any bias in the sampling of data, in the later part of the report, we also show results for simulating the model learning 100 times each with a different set of sampled data.

3.4 Data Splitting

Finally, for all the models described below, we have initial train-test split of 5%, i.e. the training data comprises 5% of the class balanced dataset. We select one sample on every round after the initial train test split.

4 Methods

4.1 Random Learning

One of the most simple and commonly used baseline for active learning is random sampling, where data points are selected for labeling at random at each round. While random sampling can be a useful baseline for comparison with other active learning methods, it is generally not as effective as more sophisticated methods that take into account the distribution of the data and the model's uncertainty. This is because random sampling does not take into account the information content of each unlabeled sample and may waste labeling efforts on easy or irrelevant examples. In this project, therefore, we will primarily use this as a means to set a baseline for all other models.

4.2 Uncertainty Sampling

In addition to random learning, we implemented least confident uncertainty sampling method for initial analysis, as it is one of the most common and effective active learning method. We expect uncertainty sampling to be effective, as it tends to work well on data with large number of features. This method can tune our model so that it focuses on specific features that are most informative for the classification task. From the pool of data, least confident method samples instances with the lowest predicted probability given the sample, using the formula:

$$x^* = \operatorname{argmax}_{x \in \mathcal{U}} 1 - P(\hat{y}|x)$$

where $\mathcal{U}$ is the pool of labeled data from which new instances are queried and $\hat{y}$ is $\operatorname{argmax}_{y \in C} P(y|x)$, where C is the list of different class labels.

4.3 Query by Committee

Query by Committee(QBC) query selection method selects the most informative instances for labeling by training numerous models through bootstrapping train data, then using the disagreement of those models of the query as uncertainty measure. Instance with the highest disagreement will be added to the train data. Logistic regression will be the base learner for our QBC method. The committee will consist of four classifiers, Decision Tree, K-Nearest Neighbor, Logistic Regression, and Gaussian Process. QBC would be a good method to try first as our data may not be linearly separable. Sampling instances in the regions with the most uncertainty would define the decision boundary well and so QBC is expected to have a better performance than our passive learning.

4.3.1 Different QBC Implementations

Within the QBC implementation, we attempted several architectures and experimented with multiple parameters to find the most accurate model. These parameters included the size of the committee (size ranged from 4 - 7), the committee classifiers, and the disagreement metrics, including disagreement rate, hard-vote entropy, and average KL-divergence.

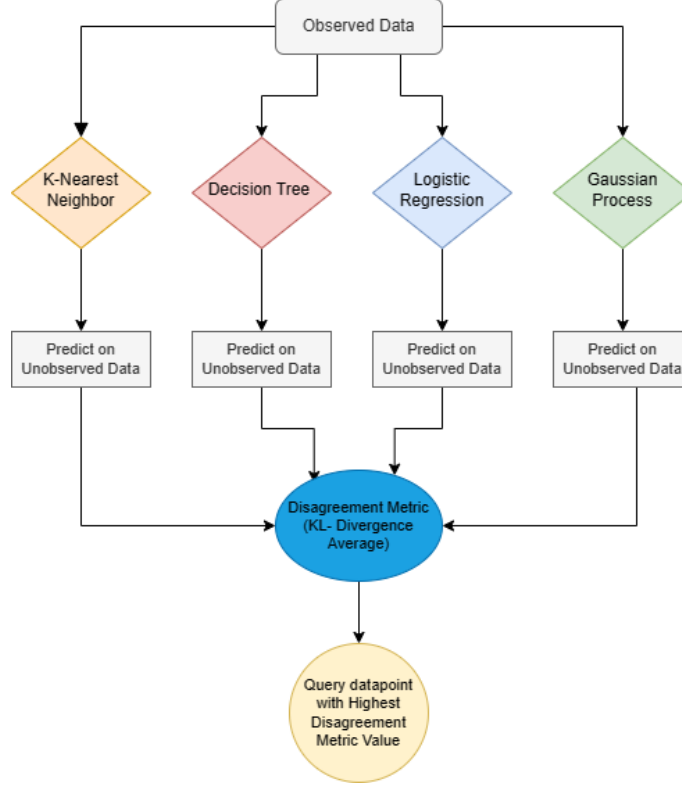


Figure 2: Model Workflow of Query by Committee Active Learning.

4.3.2 Platt Scaling

In addition to the above QBC methods, we also implemented Platt Scaling on the committee predictions. Platt scaling is a method for calibrating probabilistic outputs from a classification model. It involves fitting a logistic regression model to the outputs of the original model and treating the outputs as a continuous variable. Platt scaling is used to improve the calibration of probabilistic outputs from a model, especially when the original model is overconfident or underconfident in its predictions. By using Platt scaling, the predicted probabilities can be transformed to have better alignment with the true probabilities, thus leading to better calibrated models.

4.4 Expected Model Change

Expected Model Change(EMC) query selection method chooses the most informative instances for labeling by computing the expected change in the model's performance by the instance. Instances that will bring the largest expected change will be selected sequentially from the pool of labeled data. Our base learner is logistic regression and to calculate the expected change, the norm of the gradient of the sklearn logistic

regression loss, binary-cross entropy loss, is multiplied by the probabilities of each binary class conditioned on the new instance that is to be added to the model. Change in the gradient brought by all instances are considered and one instance that brought the largest change will be selected in each model update. Following formula describes the instance selection process:

$$x^* = \operatorname{argmax}_{x \in \mathcal{U}} \sum_{y \in \{0,1\}} P(y|x) \|\nabla L_{\Theta}(D_L \cup (x, y))\|$$

where $\mathcal{U}$ is the pool of labeled data from which new instances are queried, D_L is the classifier, ∇L_{Θ} is the gradient of the loss with respect to all the parameters and $\|\cdot\|$ is the Euclidean norm. EMC will capture the change brought to the model by each instance, so this feature could be useful for a small train-test set like in our procedure.

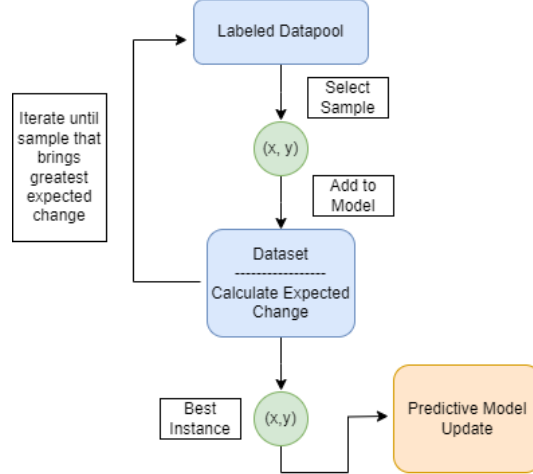


Figure 3: Model Workflow of Expected Model Change.

Due to uncertainties regarding this model, the results and discussion for this model is available in the appendix of the report.

4.5 Learning to Active Learn through Deep Reinforcement model

There are not many recorded implementations of attempting to do reinforcement learning as a prospective way to do active learning, despite its potential in this space. Many active learning heuristics rely on dataset-specific assumptions, making their performance highly dependent on the particular dataset being used. By formulating active learning as an RL problem, we can potentially overcome some of the limitations of traditional active learning heuristics that are specific to particular datasets.

The basic idea is that the RL agent selects which examples to label in order to maximize some reward function, where the reward function could be based on the improvement in classifier performance. The current implemented model was loosely inspired by Fang *et al.* [3] and is depicted in Fig.4. The formulation of this method is as discussed below.

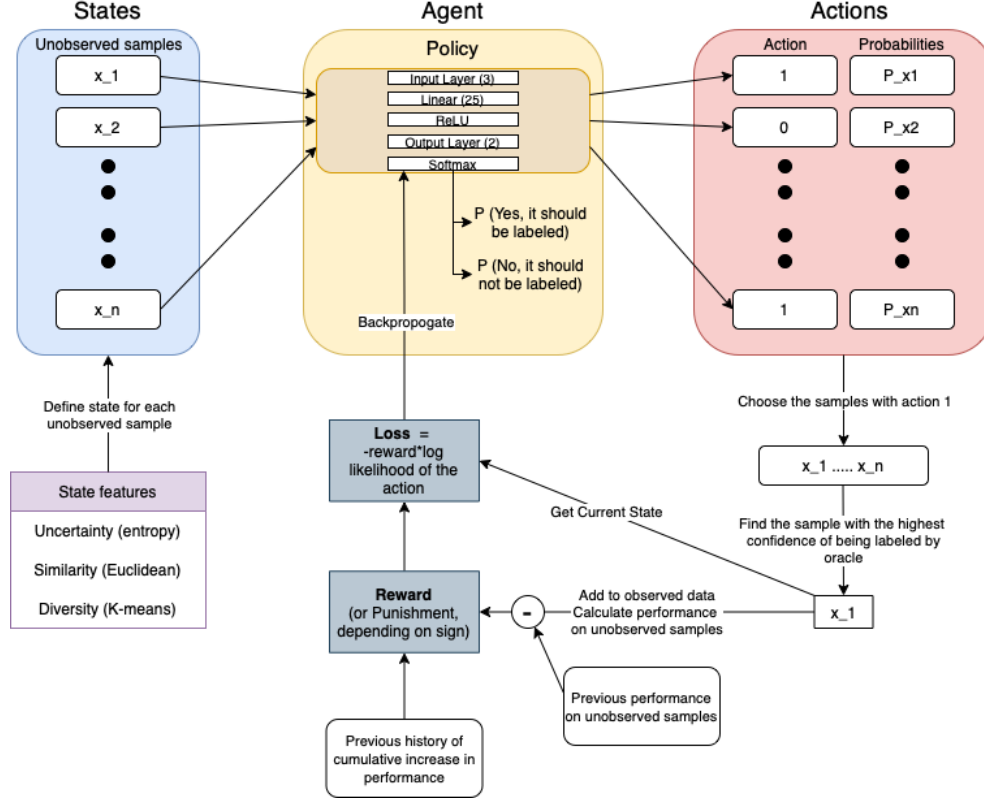


Figure 4: Model Architecture of Deep Reinforcement based Active Learning.

4.5.1 States

We aim to build a multi-state mapped to multi-action scenario at each round i . At any round i , the states are defined by what we will call a states feature matrix, i.e. we will calculate certain features for each unobserved sample j , based on metrics like Uncertainty, Similarity and Diversity in our example. If the number of classes is defined as C , the uncertainty metric was calculated using the entropy method as $H_i(j) = -\sum_{k=1}^C p_k \log_2 p_k$. This metric captures the similarity between samples, with closer samples being more similar. For similarity, we use the Euclidean distance between each unlabeled sample and other unlabeled samples. This metric captures the similarity between samples, with closer samples being more similar. Finally, diversity is

calculated by performing K-means clustering on the unlabeled samples and measuring the distance of each sample from the center of every cluster. This metric captures the diversity of the samples, with samples that are far away from the center of each cluster being more diverse. This approach allows us to select the most informative samples for labeling, based on a combination of uncertainty, similarity, and diversity. It is possible to adapt the states matrix to take more more features like representativeness and other metrics that may describe the unlabeled sample space.

4.5.2 Pool based Learning

In our approach, we assume that we have access to all the unlabeled samples in the pool for selection, on each round. We have designed our approach to be flexible, allowing us to select a single sample or a batch of samples at every round.

4.5.3 Agent

In our approach, we use a state feature matrix of size $(n,3)$, where n is the number of unlabeled samples in the pool and 3 is the number of state features in our project. The state feature matrix is passed to a policy-learning agent, which learns which sample or batch of samples should be selected on every round in order to increase the effectiveness of the model to predict the unlabeled data for selection. The policy-learning agent is implemented using a Multi-layer Perceptron (MLP) architecture. It has an input layer with size equal to the number of features in the state feature matrix, a Linear layer with 25 hidden units, and an output layer. We apply softmax activation at the output layer to obtain the probability of a particular sample belonging to either of the two classes, i.e., the probability of whether it should be selected or not selected. By learning from the state feature matrix, the policy-learning agent can determine which samples or batch of samples are most informative for labeling.

4.5.4 Actions

Using the probabilities output from the policy, the agent annotates whether a particular sample should be labeled 1 or 0, where 1 implies that an oracle should be requested to label the sample, while 0 implies otherwise. Considering that the model needs to learn how to do this efficiently, we select only those sample/s with the highest probability of having an action 1. These sample/s are then labeled by an oracle and added to

the list of labeled samples. Note that the lingo states “sample/s” here instead of referring to just one sample, since the code is adaptable to execute a batch mode.

4.5.5 Reward

In order to learn the active learning policy, an initial scalar reward is calculated. Firstly, a new testing accuracy is calculated based on the updated training data. We are also interested in seeing if in the history of testing accuracies, there has been a consecutive increase in the performance. We state that multiple jumps in accuracies should warrant a greater reward to the agent. Considering all this, our reward for round i is calculated as: $\text{reward} = (\text{new_accuracy} - \text{previous_accuracy})^{(1/\text{reward_counter})}$, where reward_counter is a counter for the number of previous consecutive increases in accuracy. We reset this to 1 if there is any instance where the new accuracy drops below previous accuracy. Note that the accuracy measures here are ranging from 0 to 1, therefore when reward_counter is greater than 1, the exponent drops to a value below 1, which in turn increases the reward measure.

4.5.6 Loss

In order to learn a good policy, we need to formulate a loss function based on the reward. In our project, we use the classic reinforce algorithm to calculate the loss as $Loss = \frac{\sum_{j \in J} -\text{reward} * \log_2 p(a=1|j)}{|J|}$, where J is the batch of selected samples, a is the action performed by a sample j in J , $p(a|j)$ is the likelihood of performing the action given the sample being selected. This loss function is back-propagated to the policy network and on consecutive rounds, the loss is minimized.

4.5.7 Training

Once this architecture is set up, training can be conducted over consecutive rounds, where at the end of each round, the agent receives a reward based on its choice of samples it has asked the experimenter to label using an oracle. With every passing round, the agent can improve its knowledge of which samples are likely to increase the performance of the model. Over multiple rounds, we propose that the agent will start to have a good understanding of which samples should be selected to achieve the best possible model performance.

4.5.8 Exploration vs Exploitation

Our model also allows us to control the probability of exploration versus exploitation occurring during the selection process. In the exploration mode, we randomly select samples from amongst the unlabeled set with action annotated as 1 by the agent. This is different from exploitation where we take the most confident action for selection. By specifying the exploration percentage, we can balance the trade-off between trying out new samples and exploiting what we already know, and find a strategy that maximizes the reward over time.

5 Results & Discussion

5.1 Comparative Study of all Models

We implemented uncertainty sampling, query-by-committee and Reinforcement-based Active Learning model in this section. The performance metrics: Accuracy, Precision, Recall (Fig.5d) and F1 score (Fig.5e) were calculated and plotted. From the accuracy plot (Fig.5b), we observe that the reinforcement based model and uncertainty sampling have almost equivalent performance in later half of the implementation. Although, the mean accuracies seem similar at this point, we do acknowledge that high standard deviation measures in uncertain sampling. The lower performance of reinforcement based method is discussed more in Section 5.2. QBC seems to have a performance that is no better than random learning. Although we are not quite sure why this is happening when uncertainty sampling has higher performance, we deduce that that using different models in QBC might be making the overall method inefficient. The precision plot (Fig.5c) seems to be following a similar trend as accuracy with respect to QBC and random sampling, we observe some difference between Reinforcement and Uncertainty sampling, which indicates that uncertainty sampling seems to be having more false positives in the testing.

Quite unlike the previous two metrics though, the Recall seems to be very high for uncertainty sampling and there is a large disparity between this and all other models. This implies that the model seems to be much better at predicting inactive molecules more-so than active molecules given high precision but low sensitivity results. Due to this huge bump in recall, the F1 scores are affected accordingly, and we observe Uncertainty Sampling having a high F1 score.

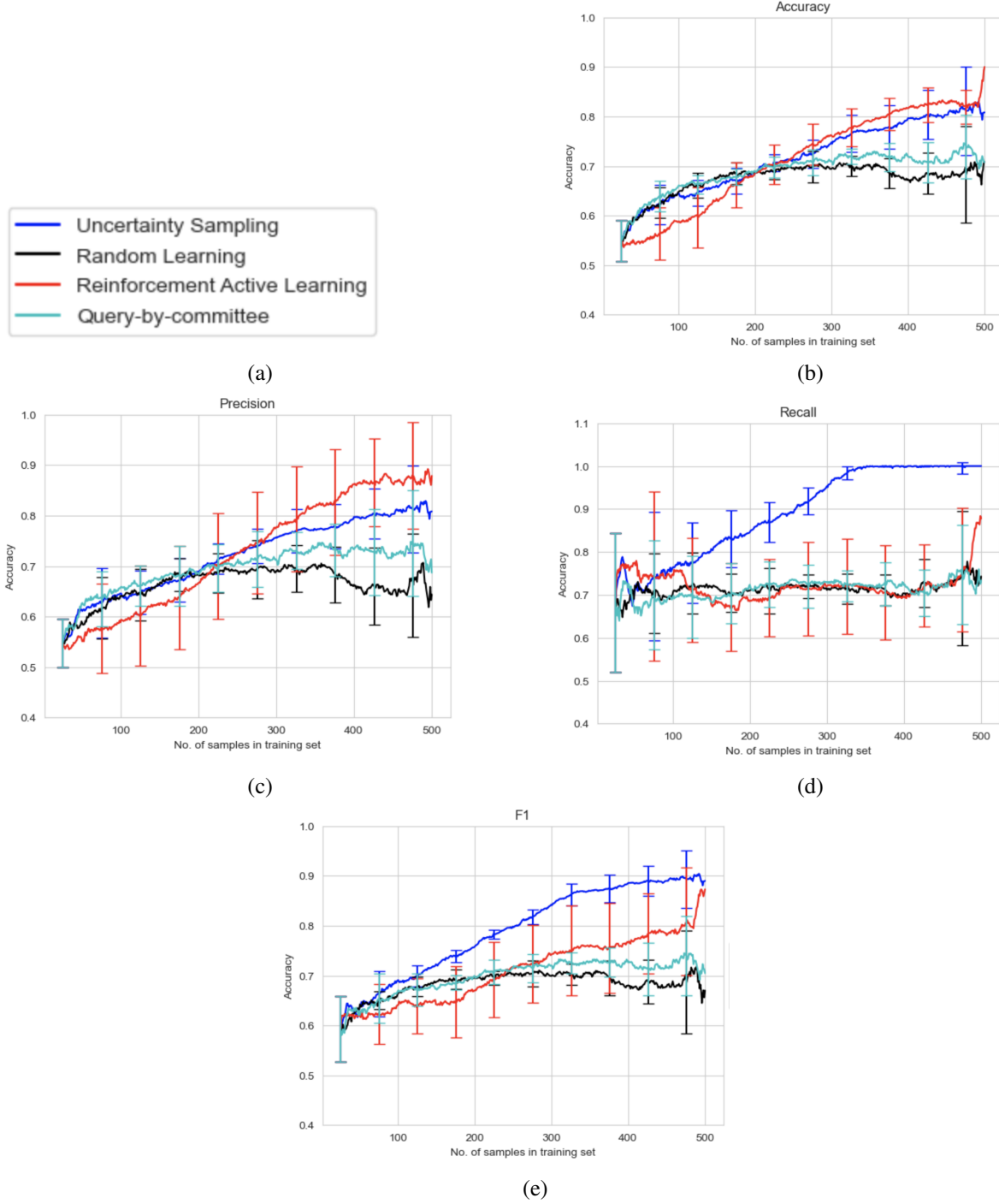
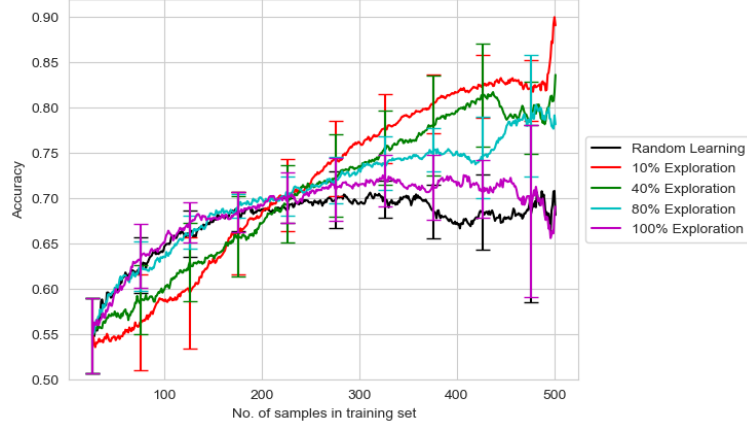


Figure 5: (a) Legend, (b) Accuracy comparison between models, (c) Precision comparison between models, (d) Recall comparison between models, (e) F1 Score comparison between models. Legend: Blue: Uncertainty Sampling, Red: Reinforcement Active Learning, Cyan: Query-by-Committee, Black: Random Sampling

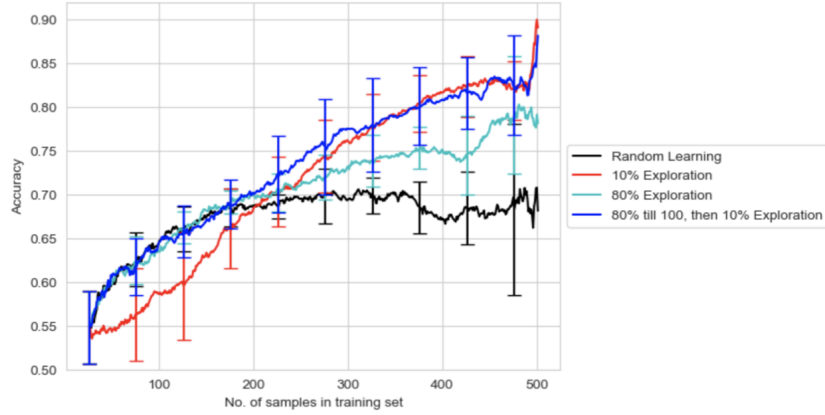
5.2 Exploration vs Exploitation Study

Exploitation-Exploration trade-off is a key point of study in both active learning as well Reinforcement models. In this section, we review the effect of varying the probability of exploration in the reinforcement learning model. We observe the trends in performance for a probability of 10, 40, 80 and 100% exploration. From the results in Fig.6a, we observe that the 10% exploitation beats all other models to get high performance after 200 samples in the training set. This is followed by 40, 80 then 100% cases, which starts leaning more towards a random sampling trend, and this is expected since doing too much exploration can result in the policy network not learning anything useful. We also observe that for a low exploration probability, the initial performance is not too great compared to other models including random learning. This is also characterized by a higher variance across simulations. Since unlike a traditional machine learning model, we do sequential training using the selected samples on each round, it is expected that the model's initial performance is lower or only as good as random learning. Although it looks like there is a visible difference between the initial random learning and RL method, the high variance in the performance is a give-away that this disparity could mostly be a random seed issue, wherein a different initial training set may produce a different initial trend (i.e. the trend upto 200 samples in the training set). Although not shown due to cluttering of images, a previous iteration of the experimental run on lesser number of simulations and a different seed demonstrated an initial trend closer to random learning.

Previous research on exploration-exploitation trade-offs in active learning has been quite successful [7, 2]. With exploration also being a key part of reinforcement learning, we are able seamlessly study this trade-off for our active learning model. Literature suggests that an initial exploration phase helps the RL models perform better. Therefore, to accommodate this hypothesis, we study the effect of 80% exploration in the initial rounds till there are 100 samples in the training set, after which we switch to a 10% exploration probability. Fig.6b proves the hypothesis well as the performance in the initial rounds have increased and is almost equivalent to random sampling. It further continues to increase much more beyond the 200th mark to become a top performer amongst all models. It is quite interesting to note that although high probability exploration stopped at 100 samples, the performance is still maintained till the 200th mark, that too with a very low variance across simulations. This could possibly be because the model is more confident about decision making due to this initial exploratory phase.



(a)



(b)

Figure 6: (a) Exploration vs Exploitation study with varying exploration probabilities, (b) Exploration vs Exploitation study with 80% Exploration until 100 samples in the training data and then switching to 10% Exploration.

5.3 Random Undersampling with 100 simulations

We recognize that our classes are from an undersampled dataset and that there could be bias in our results due to using the same undersampled data across all simulations. In order to address this issue, we performed 100 simulations, with Random Undersampling occurring on each simulation, on the Deep Reinforcement Active Learning model since it was the best performer in our comparison section. Since each time a different set of inactive-labeled sample will be used, we believe it will resolve some degree of bias that may have existed previously. The results are shown in Fig.7.

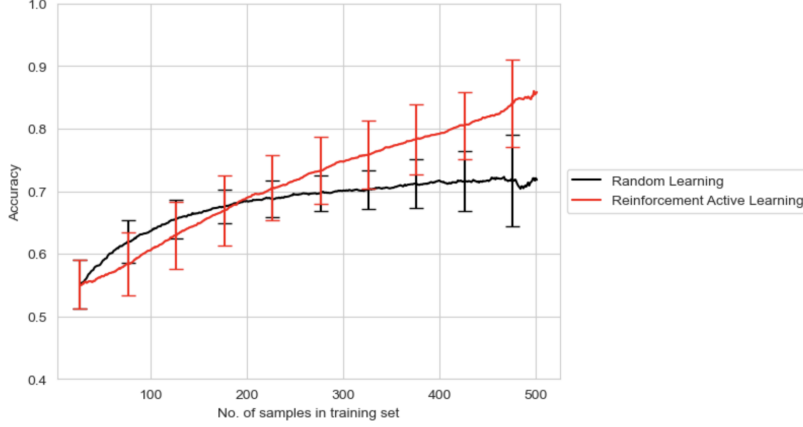


Figure 7: 100 simulations of Deep Reinforcement Active Learning with Random Undersampling on every simulation.

6 Conclusion

In the implementation of Uncertainty Sampling, we found that the model gives out lesser false negatives than false positives given the results. Considering all models and their metrics, both uncertainty sampling and reinforcement learning method seems to perform best.

In the implementation of QBC, we found that using a committee size of four, with bootstrapping, and Platt Scaling, the performance was equivalent to that of random sampling. This result was unexpected as we hypothesized that QBC would perform better than random sampling. This discrepancy could be either due to too few committee members or not enough diversity among committee members. This would result in the variance between the committee models being low and therefore the disagreement metric would also be low, resulting in incorrect querying.

The reinforcement learning model performed far better than what we thought when we first started working on it. The current state of research on the use of reinforcement learning (RL) in active learning is relatively limited, with only a few studies exploring this approach. In a brief literature survey, only two papers were found that specifically implemented this technique. Fang *et al.* [3] used RL to guide the active learning process for language modeling, while Werner *et al.* [11] applied RL to improve image classification. Despite the limited number of studies on this topic, the basic idea of using RL to guide the active learning process is promising. Further research is needed to explore the potential of this approach, particularly in fields such as

bioinformatics and drug discovery, where active learning is commonly used to identify potential targets for drug development.

7 Future Work

Our project is quite adaptable to a variety of purposes especially the novel Deep Reinforcement Active Learning method. In fact, there were more analyses that we were interested to pursue, but due to lack of time and computational resources to every member in the team, we had to push our ideas to the future work section.

- Visualization of which samples were getting selected in the pool: Initially, we were planning to visualize this by observing the B-score values of selected samples on each round. We wanted to see if samples near the threshold of -17.5 were getting selected more often, and if so, how did that affect the model performance. Calculating the sample similarity of the selected sample is also a potential way to observe the samples getting selected.
- Exploring various stopping criteria: This is a crucial part of any model training and testing, especially when a budget is involved, as it imitates a real-world scenario in any pharmaceutical company, especially so for our project motivation.
- Random Undersampling or other sampling techniques for 100 simulations or more: We tested this out with the Reinforcement-based model, but due to lack of computational resources, the other models couldn't be tested out. It would also interesting to explore how other sampling techniques affect the trends in our result.
- Implementation of class-imbalanced data: In a real world scenario, it is not unexpected to see large amounts of data having a high degree of class imbalance. We would ideally not want any bias in our models, so training them on such data would be more beneficial. But, this comes with its own limitations. We propose that with some improvements to the Reinforcement-based models, it is possible to resolve this issue.

7.1 Code Availability

The code is public and available at the following [github repository](#). It comes with a README file with a short note on how to implement them.

8 Appendix

Ten simulations were run for the EMC implementation and all simulations reached accuracy, precision and recall metrics of 1 after the model has been exposed to half of the unseen data pool. Since this has not been observed before, it was difficult to make discussions based on the outcome of the model. Some of the reasons why this could be possible might be that our data is in fact too simple and that a simple EMC was able to produce really great results. But, keeping in mind the results of the random sampling and observing how the performance of EMC neither seems to change over multiple rounds nor does it start overfitting the model, we are left befuddled on this result. Our code seems clear of all bugs and multiple unit tests have been conducted to affirm this.

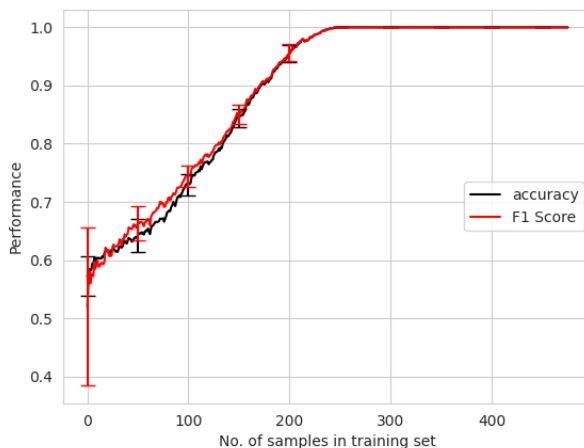


Figure 8: 10 simulations of Expected Model Change Query Method Performance

References

- [1] CARRACEDO-REBOREDO, P., LIÑARES-BLANCO, J., RODRÍGUEZ-FERNÁNDEZ, N., CEDRÓN, F., NOVOA, F. J., CARBALLAL, A., MAOJO, V., PAZOS, A., AND FERNANDEZ-LOZANO, C. A review on machine

- learning approaches and trends in drug discovery. *Computational and structural biotechnology journal* 19 (2021), 4538–4558.
- [2] CEBRON, N., AND BERTHOLD, M. R. Active learning for object classification: from exploration to exploitation. *Data Mining and Knowledge Discovery* 18 (2009), 283–299.
- [3] FANG, M., LI, Y., AND COHN, T. Learning how to active learn: A deep reinforcement learning approach. *arXiv preprint arXiv:1708.02383* (2017).
- [4] HEID, E., AND GREEN, W. H. Machine learning of reaction properties via learned representations of the condensed graph of reaction. *Journal of Chemical Information and Modeling* 62, 9 (2021), 2101–2110.
- [5] JUKIC, M., AND BREN, U. Machine learning in antibacterial drug design. *Frontiers in Pharmacology* (2022), 1284.
- [6] NAIK, A. W., KANGAS, J. D., SULLIVAN, D. P., AND MURPHY, R. F. Active machine learning-driven experimentation to determine compound effects on protein patterns. *Elife* 5 (2016), e10047.
- [7] OSUGI, T., KIM, D., AND SCOTT, S. Balancing exploration and exploitation: A new algorithm for active machine learning. In *Fifth IEEE International Conference on Data Mining (ICDM’05)* (2005), IEEE, pp. 8–pp.
- [8] RAHMAN, A. Z., LIU, C., STURM, H., HOGAN, A. M., DAVIS, R., HU, P., AND CARDONA, S. T. A machine learning model trained on a high-throughput antibacterial screen increases the hit rate of drug discovery. *PLOS Computational Biology* 18, 10 (2022), e1010613.
- [9] SELIN, C., STIETZ, M. S., BLANCHARD, J. E., GEHRKE, S. S., BERNARD, S., HALL, D. G., BROWN, E. D., AND CARDONA, S. T. A pipeline for screening small molecules with growth inhibitory activity against *burkholderia cenocepacia*. *PLoS One* 10, 6 (2015), e0128587.
- [10] STOKES, J. M., YANG, K., SWANSON, K., JIN, W., CUBILLOS-RUIZ, A., DONGHIA, N. M., MACNAIR, C. R., FRENCH, S., CARFRAE, L. A., BLOOM-ACKERMANN, Z., ET AL. A deep learning approach to antibiotic discovery. *Cell* 180, 4 (2020), 688–702.
- [11] WERNER, T. Reinforcement learning approach to active learning for image classification. *arXiv preprint arXiv:2108.05595* (2021).